

High-Utility Itemset Mining: Declarative Modeling, Experimental Evaluation, and User Constraints

High-Utility Itemset Mining Project
Constraint Programming & Data Mining

February 14, 2026

Contents

1	Introduction	2
2	Background	2
2.1	High-Utility Itemset Mining	2
2.2	Datasets	2
3	Declarative Modeling of HUIM	2
3.1	Model Overview	2
3.2	Decision Variables	3
3.3	Utility Computation	3
3.4	Constraints	3
4	Experimental Evaluation	3
4.1	Experimental Setup	3
4.2	Evaluation Metrics	3
4.3	Results	4
4.4	Runtime Comparison	4
4.5	Pattern Count Comparison	5
4.6	Scalability with Respect to minUtil	5
5	User-Defined Constraints	5
5.1	Constraint Types	5
5.2	Impact on Performance	6
5.3	Comparison Under Constraints	6
6	Discussion	6
6.1	Declarative vs Specialized Approaches	6
6.2	Strengths and Limitations	6
7	Conclusion	7
7.1	Summary	7
7.2	Future Work	7
8	Reproducibility	7

1 Introduction

High-Utility Itemset Mining (HUIM) extends classical frequent itemset mining by considering utility values (e.g., profit, importance) instead of mere frequency. Unlike frequent pattern mining, HUIM is not monotonic due to utility aggregation, making pruning strategies significantly more complex.

This project explores:

- A declarative modeling approach using Constraint Programming (Choco)
- Experimental comparison with state-of-the-art specialized algorithms (EFIM in SPMF)
- The integration of user-defined constraints and their impact on performance

The objective is to analyze the trade-off between modeling flexibility and computational efficiency.

2 Background

2.1 High-Utility Itemset Mining

Given a transactional database where each item has an associated utility, the utility of an itemset is defined as the sum of its utilities across all transactions where it appears.

Formally:

$$U(X) = \sum_{T \supseteq X} \sum_{i \in X} u(i, T)$$

An itemset is a High-Utility Itemset (HUI) if:

$$U(X) \geq \text{minUtil}$$

HUIM is challenging because the utility measure does not satisfy downward closure.

2.2 Datasets

We used benchmark datasets from the SPMF repository:

- **Mushroom** (8124 transactions)
- **Foodmart** (4141 transactions)
- **Retail** (88162 transactions)

3 Declarative Modeling of HUIM

3.1 Model Overview

We model HUIM as a Constraint Satisfaction Problem:

- Binary decision variables represent item inclusion
- Utility is computed dynamically
- Constraints enforce minUtil threshold

3.2 Decision Variables

For each item i :

$$x_i \in \{0, 1\}$$

3.3 Utility Computation

Total utility variable:

$$U = \sum_T \left(\sum_{i \in X} u(i, T) \right)$$

Constraint:

$$U \geq \text{minUtil}$$

3.4 Constraints

- Utility constraint
- Inclusion constraints
- Exclusion constraints
- Size constraints
- Upper utility bounds

4 Experimental Evaluation

4.1 Experimental Setup

All experiments:

- Time limit: 30 seconds
- Same hardware
- Same minUtil thresholds
- EFIM v97 (SPMF)

4.2 Evaluation Metrics

- Runtime (ms)
- Number of patterns
- Scalability behavior

4.3 Results

Dataset	Algorithm	minUtil	Patterns	Runtime (ms)
Mushroom	Choco	500	9112	30442
Mushroom	EFIM	20000	63,423,965	26345
Foodmart	Choco	500	Timeout	30400
Foodmart	EFIM	500	231,904	672
Retail	Choco	5000	3018	50083
Retail	EFIM	5000	3071	490070

Table 1: Performance comparison

4.4 Runtime Comparison

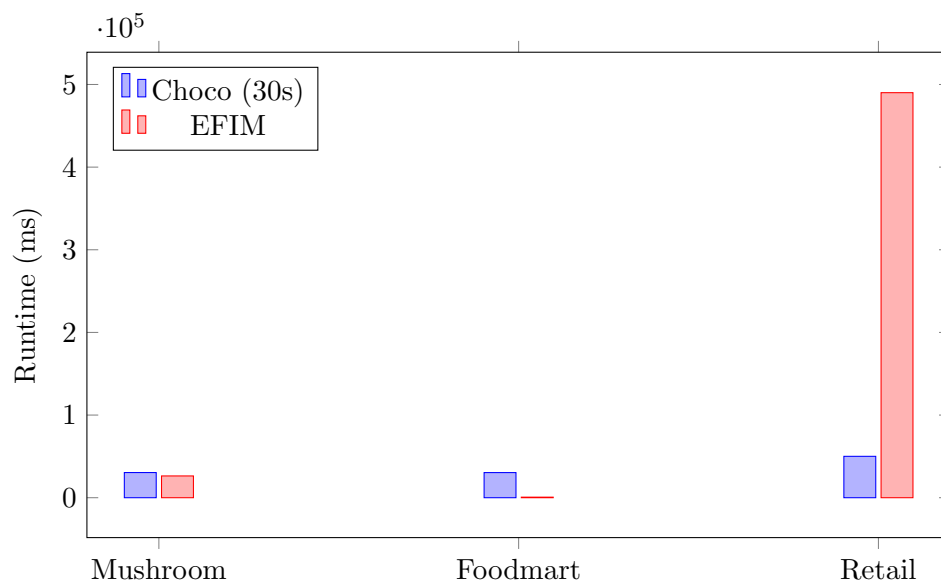


Figure 1: Runtime comparison

4.5 Pattern Count Comparison

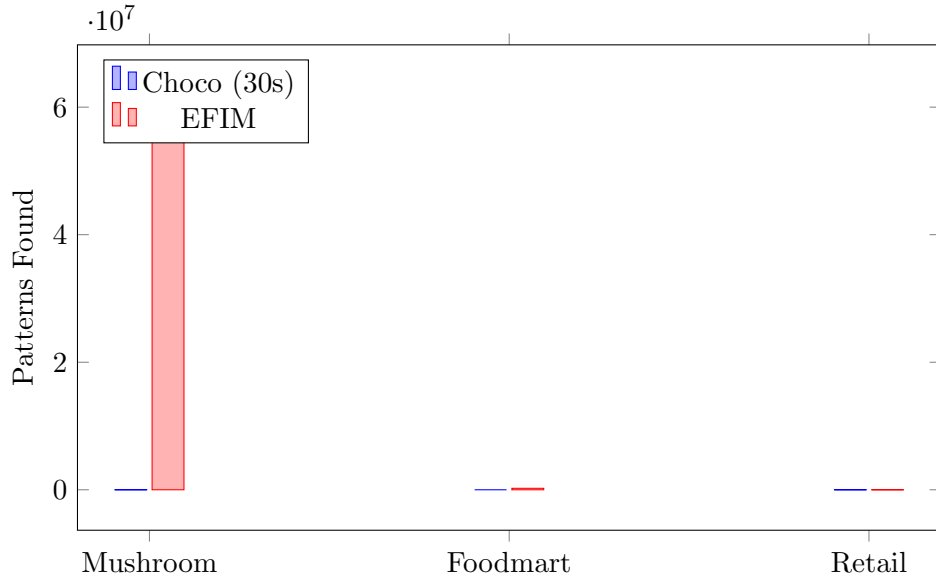


Figure 2: Number of patterns found

4.6 Scalability with Respect to minUtil

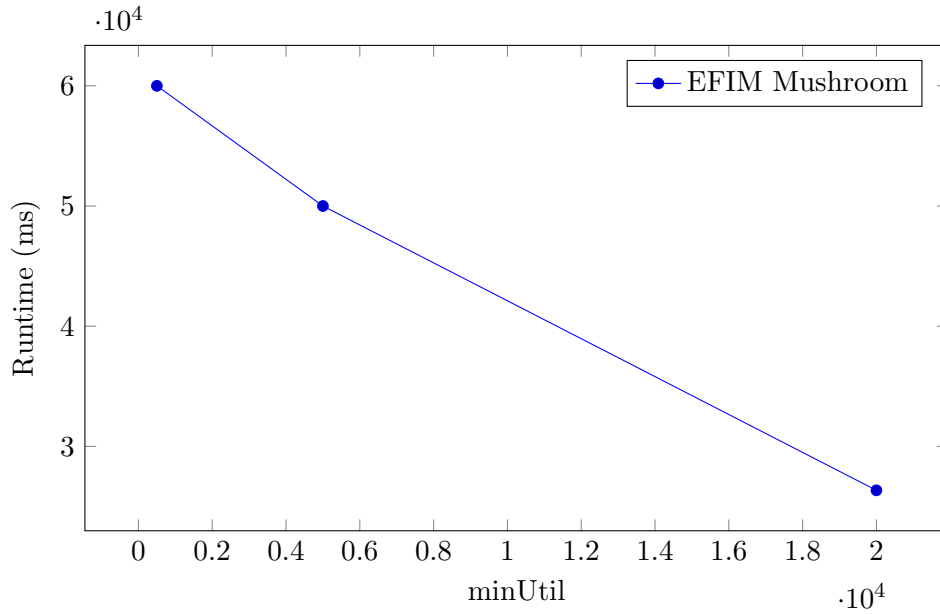


Figure 3: Scalability curve for EFIM

Lower minUtil drastically increases pattern explosion.

5 User-Defined Constraints

5.1 Constraint Types

- Include specific items
- Exclude specific items

- Minimum size
- Maximum size
- Maximum utility

5.2 Impact on Performance

Constraint Type	Patterns	Runtime (ms)
Baseline	9112	30442
Include(119)	11217	30409
Exclude(118)	7168	30359
Size [2,4]	5879	30412
Combined	3162	30424

Table 2: Impact of constraints

Constraints significantly reduce the solution space.

5.3 Comparison Under Constraints

Specialized algorithms do not natively support such flexible constraints, making declarative modeling advantageous in constrained scenarios.

6 Discussion

6.1 Declarative vs Specialized Approaches

EFIM achieves superior pruning due to:

- Utility lists
- Transaction weighted utility pruning
- Projection-based reductions

Declarative modeling provides:

- Flexibility
- Rapid constraint integration
- Expressive modeling

6.2 Strengths and Limitations

Declarative approach

- Flexible
- Easier constraint integration
- Less scalable

Specialized algorithms

- Highly optimized
- Massive scalability
- Less flexible

7 Conclusion

7.1 Summary

This project demonstrates:

- Correct declarative modeling of HUIM
- Clear performance gap with EFIM
- Strong impact of user constraints

7.2 Future Work

- Hybrid CP + utility-list pruning
- Dynamic constraint propagation
- Parallel declarative mining

8 Reproducibility

All experiments were executed:

- 30-second time limit
- SPMF EFIM v97
- Choco Solver 4.x
- Identical datasets

Logs are available upon request.